Revisiting Challenges for Selective Data Protection of Real Applications

Lin Ma
linma@zju.edu.cn
Zhejiang University

Jinyan Xu
phantom@zju.edu.cn
Zhejiang University

Jiadong Sun
simonsun@zju.edu.cn
Zhejiang University

Yajin Zhou*
yajin_zhou@zju.edu.cn
Zhejiang University

Xun Xie
xiexun@zju.edu.cn
Zhejiang University

Wenbo Shen
shenwenbo@zju.edu.cn
Zhejiang University

Rui Chang
crix1021@zju.edu.cn
Zhejiang University

Kui Ren
kuiren@zju.edu.cn
Zhejiang University

Abstract

Selective data protection is a promising technique to defend against the data leakage attack. In this paper, we revisit technical challenges that were neglected when applying this protection to real applications. These challenges include the secure input channel, granularity conflict, and sensitivity conflict. We summarize the causes of them and propose corresponding solutions. Then we design and implement a prototype system for selective data protection and evaluate the overhead using the RISC-V Spike simulator. The evaluation demonstrates the efficiency (less than 3% runtime overhead with optimizations) and the security guarantees provided by our system.

CCS Concepts: • **Security and privacy → Hardware security implementation**.

Keywords: selective data protection, tag architecture

ACM Reference Format:
Lin Ma, Jinyan Xu, Jiadong Sun, Yajin Zhou, Xun Xie, Wenbo Shen, Rui Chang, and Kui Ren. 2021. Revisiting Challenges for Selective Data Protection of Real Applications. In *12th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '21), August 24–25, 2021, Hong Kong, China.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3476886.3477504

*Corresponding author.

APSys '21, August 24–25, 2021, Hong Kong, China

ACM ISBN 978-1-4503-8698-2/21/08…$15.00
https://doi.org/10.1145/3476886.3477504

1 Introduction

Selectively protecting sensitive data is a promising technique to defend against the data leakage attack. Some recent systems [5, 6, 23, 33] implement this type of protection to improve the performance by only protecting the sensitive data instead of all memory objects. To achieve this, they require the developer to annotate variables that may contain sensitive data and then use the static analysis tool to find all potential candidates, e.g., variables copied from the sensitive data, that need to be protected. DataShield [6] and ConfLLVM [5] prepare dedicated memory regions and additional bound checking for potentially sensitive variables. Ginseng [33] protects sensitive data by always putting it into registers except when task switching or interrupts occur.

Although these approaches are different in details, they all use the static data flow analysis to find out possible memory locations that may hold the sensitive data. Then they instrument all load and store instructions that could operate on these locations to achieve the protection. However, this methodology suffers from the following shortcomings. First, the static analysis has a precision issue in the points-to analysis [34] when locating sensitive data. Second, the protection granularity is coarse-grained. When protecting a sensitive candidate inside a data structure, they need to mark the entire structure as sensitive, introducing unnecessary performance overhead.

To resolve these shortcomings and achieve practical protection, we introduce `Conch`, a solution to selectively safeguard the confidentiality of data against diverse data leakage attacks. *The goal is to ensure that the sensitive data is never exposed to untrusted user-space memory as plaintext for its entire lifetime.* To this end, `Conch` leverages the dynamic information flow tracking [7] technique supported by underlying hardware (tagged architecture) to offer precise selective data protection, thus solving the imprecise static point-to analysis problem. Also, the protection is in the machine-word-level granularity, which is more fine-grained than the

structure-level granularity. Besides, Conch utilizes the hardware supported in-memory cryptographic transformation with per-thread encryption keys to offer robust confidentiality protection. With the protection of our system, the attacker cannot spoil the confidentiality of the sensitive data even under a strong threat model since the leaked data is encrypted.

However, when applying this solution to real applications, we need to confront three challenges *that were usually neglected by previous systems*. First, the sensitive data can be from external sources such as a file. The data could be stored by the OS kernel into a user-provided buffer when using the system call (e.g., the `read` system call) to read the content from the file. Thus, a mechanism is needed to mark the sensitive data inside the buffer. Second, the granularity conflict between the hardware tag and the memory manipulation instructions (`load` and `store`) needs to be solved. Otherwise, the tag for sensitive data could be cleared, compromising the security guarantees of sensitive data protection. Third, the sensitivity conflict will cause semantic compatibility of the program. We need to find a solution to maintain the compatibility, while at the same time, cannot introduce the security loophole that could be abused by the attacker to bypass our protection.

We implement a system prototype on the RISC-V simulator with new instructions, tagged registers, and cache. We also implement the tag prorogation inside the CPU pipeline. We use the mibench [15] and the real-world applications to analyze the performance overhead when protecting sensitive data. After that, we perform a security analysis to evaluate the security guarantees provided by our system. The result shows that our system can safeguard the confidentiality of sensitive data with a performance overhead that is less than 3%.

Contributions In summary, our work makes the following main contributions.

- We propose and implement a system to efficiently safeguard the confidentiality of the sensitive data against diverse data leakage attacks with a strong threat model.
- We summarize three technical challenges that were usually neglected by previous systems when applying selective data protection to real-applications, and propose corresponding strategy to confront them.
- We evaluate the overhead and security guarantees provided by our system. The result shows that Conch can successfully defend against sensitive data leakage attacks with an appropriate performance overhead.

2 Background

2.1 Sensitive Data Leakage Attack

The attack uses vulnerabilities in programs to exfiltrate sensitive data from the memory. In the following, we present typical scenarios that lead to sensitive data leakage.

```
1    // p is the pointer to incoming packet
2    n2s(p,payload);
3    pl=p;
4    buffer=OPENSSL_malloc(1+2+payload+padding);
5    bp=buffer;
6    *bp++=TLS1_HB_RESPONSE;
7    s2n(payload,bp);
8    memcpy(bp,pl,payload); // over-read at pl!
9    r = ssl3_write_bytes(s,TLS1_RT_HEARTBEAT,buffer,3 +
         payload+padding);
```

Figure 1. The vulnerable `tls1_process_heartbeat()` function in OpenSSL version 1.0.1f.

Format String Vulnerability Attackers could exploit the format string vulnerability [25] to gain arbitrary memory read and write primitive. Even though many tools are presented to detect this vulnerability, it is still exists in real-world applications [1, 2] nowadays.

Buffer Over-read It is a type of vulnerability where a program, while reading data from a buffer, overruns the buffer's boundary and reads (or tries to read) adjacent memory [29]. For instance, the HeartBleed vulnerability allows the attacker to over-read around 64KB data from the memory buffer adjacent to the allocated buffer for the heartbeat packet. Besides, existing exploits use vulnerabilities like the type confusion, off-by-one, and the integer overflow to corrupt the metadata, especially the length of a dynamic object to achieve the buffer over-read primitive.

2.2 Tagged Architecture

The tagged architecture, proposed in 1973 [12], equips the machine memory with additional tags. Since its debut, it has been leveraged for security hardening, e.g., dynamic information flow tracking (DIFT) [11, 17, 18, 22, 28, 30], memory safety [13, 32], instrumentation and debugging [14], capability protection [19, 32], and others [9, 26, 27]. For example, Minos [8] uses one-bit tags to indicate the integrity of code pointers. CHERI [32] also uses one-bit tags to decide if an address stores a valid capability. The Morpheus [13] uses two-bit domain tags to distinguish between code, code pointers, data pointers and other data. Our work also leverages the tagged architecture for runtime sensitive data tracking.

3 Motivating Example and Threat Model

The HeartBleed Vulnerability The HeartBleed (CVE-2014-0160) is a serious vulnerability in the OpenSSL library. It is a buffer over-read bug in the implementation of the heartbeat extension, which allows the attacker to leak sensitive data from the remote server. The code snippet in Fig. 1 shows the vulnerable function (`tls1_process_heartbeat()`).

Specifically, the vulnerable function allocates a buffer (line 4) with the payload size (`payload`) extracted from the heartbeat request packet (line 2), which is controlled by the attacker. It finally constructs the heartbeat response packet using this buffer (line 6 to 9). The attacker can craft a malicious heartbeat request packet with a large value at the

`payload` field while only appending a small size of payload. Despite the size of the buffer for storing the actual requesting packet (what `pl` points to) is much less than the value of the `payload`, the memory copy is still executed, resulting in the buffer over-read bug. As a result, the sensitive data in memory could be leaked to attacker. To defend this, our system ensures that sensitive data is *always encrypted* in the memory, with the encryption key maintained by the kernel. The vulnerability may still exist. However, it cannot be exploited to leak the private data in the plaintext. The basic idea is straightforward. However, making it work for real applications needs to solve three challenges, which we will discuss in Section 5.

Threat Model and Assumptions Our system assumes a strong threat model that the attacker can read arbitrary use-space memory. This can be achieved through the nature of the vulnerability (the HeartBleed). However, the attacker cannot inject code into execution due to the availability of DEP. Also the attacker cannot hijack the control flow of the program. These assumptions align with previous works [5, 6, 23, 33]. We do not consider the exploit of kernel vulnerabilities to leak user-space memory. Also, the side-channel attack and the cold-boot attack are out of the scope.

4 Overall System Design

Our work aims to *selectively protect sensitive data by ensuring that it is never exposed to untrusted user-space memory as plaintext ever since its initialization.* To this end, our system requires the developer to annotate the source of sensitive data. Then the compiler wraps the source with new CPU instructions to enable tag initialization. When the program executes on the CPU, it utilizes the tagged architecture to dynamically track the propagation of the sensitive data inside the registers and caches. Before any tagged data being written into the memory, an encryption engine will transparently encrypt the data. As a result, the sensitive data will never be leaked as plaintext in the user-space memory. In the following, we will illustrate the main steps of our system.

Annotating Sensitive Data `Conch` selectively tracks and encrypts the developer-specified sensitive data. By focusing only on this subset, the performance overhead can be limited. Moreover, this human-in-the-loop strategy can be flexible and accurate compared to an automatic one [11].

Most of the previous systems [5, 6, 33] take the type-based annotation and allow the developer to mark the definition of memory objects in the source code to regard them as *sensitive*. For instance, ConfLLVM allows the developer to annotate the top-level definitions, i.e., global variables and function signatures, while Ginseng only allows the annotations of local variables. However, the type-based annotation cannot support fine-grained protection because the granularity of the sensitive data remains in the data structure level. For `Conch`, developers just need to add annotations at the sensitive data starting point, aka, *sensitive sources*. The compiler will automatically generate the machine instructions to mark the data as sensitive (using the memory tag.) Though the idea of annotating is rather simple, it is not trivial to implement because the sensitive data can come from multiple sources, such as files in the disks, inputs from keyboards, and random bytes from the pseudo-random number generator. The challenge will be discussed in Section 5.

Propagating Tags In our system, a one-bit tag is applied to a machine word. As a result, it only imposes 1.56% memory space overhead on modern 64-bit architecture. The memory accesses will be split into the data access and the tag access. To decrease the incurred extra DRAM traffic overhead, the tag cache optimization is adopted, which is proven to be highly useful [16, 27].

The tagged architecture is similar to systems for information flow tracking [4, 13]. The architecture associates each data with its tag in both the execution pipeline and the memory hierarchy. To offer robust protection, it introduces tag (or taint) propagation rules that enable a lifetime tracking for the sensitive data as well as its transformed variants. The rules strictly ensure all operations that involve sensitive data should propagate the sensitivity to the result. When the CPU executes a memory unrelated instruction, the tag of the source register may propagate to the destination register, according to the propagation rules. For memory related instructions such as `store` and `load`, the tags should be stored into, or loaded from the memory together with the data. To facilitate this process, our system augments the registers and caches to hold the tags.

Encrypting Data Our system encrypts the sensitive data (tagged data) before being written into the memory. With the memory tag, the encryption engine is transparent to the program as it does not require any instrumentation to distinguish sensitive data from others.

Our system utilizes a strong block cipher named QARMA [3]. Compared with the commonly used AES, the major advantage of QARMA is that it enables an additional input named `tweak` to parameterize the permutations. For each memory word, our system chooses its address as the tweak used in the encryption. Hence the same sensitive data in different addresses will have different encrypted outcomes, which increase the difficulty of cryptanalysis.

Our system has fine-grained management of the encryption keys by the operating system kernel. Once the system is booting, each CPU processor randomly generates a master key. After that, when a new thread is created, a per-thread key will be generated using this master key and then be associated with the context of this thread. Our system offers two additional micro-architectural registers that cannot be accessed from user-space to store the master key and the currently in-use thread key. The master key register preserves

the master key ever since its initialization. The thread key register, however, keeps changing as the thread is actively scheduling. During the context switch, the old thread's key will be saved on the kernel stack, and the current thread's key is loaded. With this fine-grained key management, Conch provides a trusted guarantee to the robust encryption defense.

5 Practical Challenges

Though the architecture is straightforward and similar to the previous one [13], applying such a defense on real programs needs to confront several challenges (These challenges were usually ignored by previous systems [13, 23, 24]. In this section, we will present the details of these challenges and the coping strategies.

5.1 Sensitive Input Channel

The sensitive data is not allowed to be exposed in the user-space memory without encryption. However, the sensitive data may already reside inside the memory before the developer has the chance to annotate it, e.g., written by the system call in the OS kernel. In other words, our system should pay attestation to every possible channel through which the sensitive data enters into the user-space memory (*Sensitive Input Channel* in this paper).

Previous systems either omit this problem [23] or require the developer to rewrite the code and ensure the buffer that receives the sensitive data is protected via isolation [6], which requires additional engineering effort. Our system proposes a design of sensitive input channel that can associate the tag to the sensitive data and initialize the defense before the data being written into userspace memory. The construction of this channel is not trivial due to the following two reasons. First, the sensitive data may come from various sources, such as files in the disks, keyboards, network sockets. Second, the sensitive input channel should not change the way (or the APIs) that are used to maintain the program's compatibility.

Specifically, `Conch` leverages the OS kernel to build the sensitive input channel. This is because the kernel is in charge of processing the data from the sensitive source and placing it into the supplied user-space memory buffer. However, the kernel has no idea whether the incoming buffer contains sensitive data or not. A direct solution is to use dedicated devices for sensitive inputs like Ginseng [33], which needs extra UART devices. However, it does not satisfy the issues of multiple types of input sources.

Our system solves the issue by allowing the developers to inform the kernel that the passing buffer will contain sensitive inputs. To accomplish this, it patches existing system calls in the kernel. For instance, a file is opened with the specified `O_SENSITIVE` flag so the kernel can return a file descriptor associated with the additional attribute. When the program reads data from this special file descriptor, the kernel can switch to the sensitive input channel and initialize the protection before the data is placed into the user-space buffer.

We argue that the design choice of `Conch` is reasonable and will not cause overprotected for two reasons. First, due to our observations, we found that real applications, like OpenSSL, will store sensitive data, like the private key, in a dedicated file without mixing with other insensitive content. Second, even there are some data, like the constant prefix for a private key, is non-sensitive, it's better to protect it and the sensitive data as a whole to prevent attackers from easily locating the sensitive data in memory.

In the implementation, `Conch` patches the `copy_to_user()` function, which is an interface that transmit the data from kernel-space to user-space memory. This function will check if the file descriptor is opened with the `O_SENSITIVE` flag to perform the actual data transmission. Besides, our system changes the `getrandom` system call, which is widely used in cryptographic algorithms, to give random value that is protected with tags.

5.2 Sensitivity Conflict

Another challenge is the sensitivity conflict that will cause the behaviors of program to not match the expectation. One example is when protecting the session key of the SSL/TLS handshaking process in the OpenSSL library, the content that is encrypted using this session key (in the `tls1_enc()` function) will be tagged as sensitive, due to the tag prorogation. Thus, it will be encrypted again by `Conch` when writing into the memory before being sent to the client. Hence, the received data cannot be decrypted since the client does not have the encrypted key maintained by `Conch`.

To solve this challenge, we provide an instruction that can remove the memory tag of a buffer. The developer can insert this instruction to the program when the data is already protected (encrypted) and needs to be shared with another entity (SSL client for instance.) However, we need to ensure that this instruction cannot be abused by the attacker to remove the tag of arbitrary memory. Fortunately, the control flow of the program cannot be hijacked (See the threat model in Section 3), thus the attacker cannot redirect the control flow to remove the tag of a buffer controlled by the attacker.

5.3 Granularity Conflict

`Conch` has one-bit tag for a machine word. This may introduce the granularity conflict between a memory tag and the byte granularity of the memory load and store instructions. We use the example in Figure. 2 to demonstrate this issue. As the figure shown, there are two variables (a 4-byte array `buf` and 4-byte integer `i`) placed into one machine word by the compiler. The code **a** reads four bytes sensitive data from a dedicated file descriptor into the `buf` so the memory tag will be set to 1 because the associated machine word contains

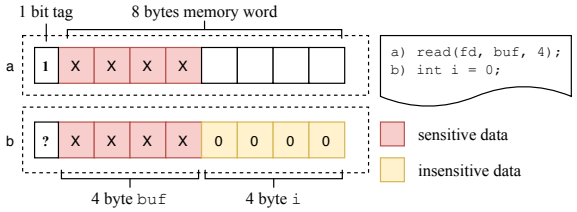

Figure 2. The granularity conflict: The dotted frames represent the state of memory word state after the a and b lines of code. **?** marks the uncertain tag value.

sensitive data. However, after executing the code **b** which puts a constant value into the variable `i`, the ambiguity about the value of the tag of this memory word is introduced since the machine word now contains both the sensitive data and normal data.

The direct solution for this conflict is to extend the tagged architecture to support per-byte memory tags and ensure one tag will be necessarily associated with exactly one variable, However the per-byte tags leads to a leap of memory overhead from the 1.56% to 12.5% to store the tags. Though there is architecture [9] claims to support unbounded bits of tag, most tagged architectures [19, 28, 30, 32] utilize only one-bit tags to reduce the memory overhead.

After surveying the recent works that utilize the memory tag for security, we sadly find that this conflict is rarely discussed. Some researches [13, 28] only focused on word aligned objects, like pointers. Nick's work [24] adopts the tag to protect data in the stack. But they only discuss word-aligned variables and ignore the fact the unaligned variables could exist.

`Conch` firstly discusses the impact of granularity conflict. To ensure the sensitive data is always protected, we decide to retain the tag and according to our evaluation (Section 7.4), the extra overhead from over-tagging is acceptable.

6 System Implementation

We have implemented a prototype system based on the Spike simulator by extending the pipeline and registers to construct the tagged architecture. Because the vanilla Spike lacks the simulation of cache, we write our own cache module plugin to simulate the instruction and data cache. We implement the extended instructions, based on the RISC-V custom instructions support, into the Spike and modify the v9.2.0 GCC compiler for support. Besides, we implement the QARMA algorithm [3], i.e., QARMA$_5$-64-σ_1, which encrypts 64-bits block data with a 128-bits key in 12 rounds. To accurately model the memory system and assess the overhead of tag accessing, we utilize the state of art memory simulator DRAMSim3 [20]. Last, we add the key management and the designed sensitive input channels in the v5.4.7 Linux kernel.

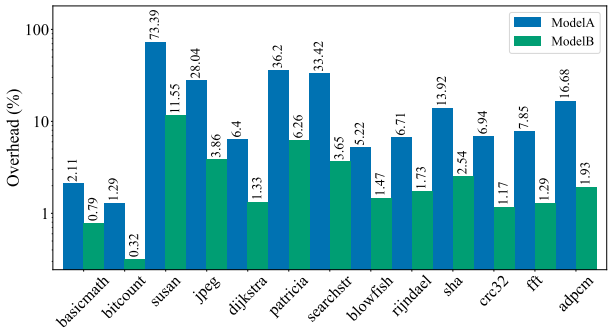

Figure 3. The runtime overhead of Conch Model A and Model B (with tag cache support) for mibench [15]

7 Evaluation

7.1 Methodology

We adopt a methodology used in TIMBER-V [31] to estimate the system runtime performance overhead by mapping executed instructions into actual cycles using different pipelined CPU models. To this end, we rewrite the histograms module in the Spike simulator to allow the precise recording of all executed instructions and trace all memory accesses. Specifically, we first define a baseline model without the tagged memory extension and then compare with another two models, one of which is equipped with tag cache optimization and another one is not. Different from TIMBER-V, which assigns cycles for each instruction empirically, we build a more accurate pipelined model and configure the CPU and cache referring to the SiFive CPU manual [35]. We additionalhy utilize the memory simulator DRAMSim3 and add the tag cache into Spike to evaluate the memory access overhead.

Baseline CPU Model As a baseline, we configure the simulator referring to the Freedom FU540-C000 Soc. Based on that, we model the instructions cycle according to the SiFive manual. For the memory `load` and `store` instructions, they cost two or three cycles depending on operand values when the cache hits. When the cache misses, we model the latency according to the DRAMSim3 simulator.

Conch CPU Model There are two models, namely Conch Model A and Conch Model B. The Model B is optimized with tag cache while A is not. For the encryption engine of these two models, we assume a four-cycles latency for QARMA$_5$-64-σ_1 to encrypt or decrypt a block, similar to the previous work [13, 21]. For the optimized Model B, we configure the 4KB eight-way associative tag cache.

We claim that the methodology is pessimistic guided because the decreased CPU cycles of register instructions will relatively increase the impact caused by the memory instructions and results in a higher overhead of tagged architecture. That is, the result can not be viewed an upper bound on the achievable performance.

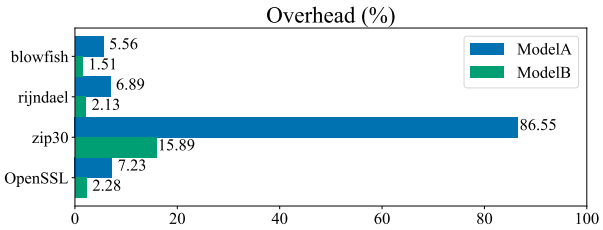

Figure 4. The runtime overhead of Conch Model A and Model B (with tag cache support) in real-world application when protecting sensitive data.

7.2 Benchmarks

To estimate the cost caused by the tagged architecture, we use the mibench [15] to measure the overhead. We use 13 programs that can be built for the RISC-V architecture without modification with the default optimization level. Since we do not annotate any sensitive data, the overhead is all caused by the tag architecture. The result is shown in Fig. 3. This result shows an average runtime overhead of 18.3% for the Model A with the maximum value of 73.39% for the program susan, an image processing program that performs large amounts of memory accesses. Other programs with frequent memory access have a comparatively high overhead compared with the CPU-intensive programs. However, with the tag cache optimization, the result (2.914% on average) is promising. It shows that the tag cache can efficiently decrease the runtime overhead of tagged architecture.

7.3 Real Programs

In this section, we use `Conch` to protect the sensitive data in real-world programs and analyze the performance overhead. We choose the cryptographic applications including blowfish and rijndael from the mibench as targets. Besides, we select the zip30 application and the OpenSSL library to enrich the evaluation. The overall result is shown in Fig. 4.

Cryptographic Algorithms To protect the sensitive data (encryption keys) of the cryptographic algorithms blowfish and rijndael, we annotate the source code to ensure that the password, which is involved in the generation of the encryption key, is protected once it enters the memory. We rewrite these two programs to read the password from the sensitive file with the `O_SENSITIVE` flag. As the Fig. 4 show, our system has low performance overhead. For the Model A, `Conch` imposes 5.56% and 6.89% overhead for blowfish and rijndael, respectively. For Model B, the overhead lower to 1.51% and 2.13%.

Cryptographic Applications We use the zip30 and the OpenSSL as target applications. The first one is a widely used compression utility, which implements a protection to encrypt the file content using a user-provided password. To protect the sensitive password, we rewrite the program to load the password from a specialized TTY input channel. The performance overhead is 86.55% for Model A and 15.89% for the optimized Model B, respectively. This relatively high overhead comes from the high frequency of memory accesses as it trivially reads uncompressed input data chunks and writes encrypted output to memory.

Table 1. The result of granularity conflict of 4 applications. In the table, we show the percentage of sensitive data that has been over-tagged and extra runtime overhead for each application.

	blowfish	rijndael	zip30	OpenSSL
Over-tagging	0.033%/0.14%	0%/0%	0.13%/1.04%	5.12%/2.16%

For the OpenSSL, we use `Conch` to protect its SSL/TLS communications. Besides the sensitive master key and the session key, we also protect the private key used in handshaking. To do this, we rewrite the `SSL_CTX_use_PrivateKey_file()` function to allow it to read the sensitive private key from the specialized channel our system supplies. For the master key and the session key, they are both obtained from the pseudo-random generator function `tls1_PRF()`. Hence, we annotate this sensitive source to initialize the protection. The result (7.23% for Model A and 2.27% for Model B) shown in Fig. 4 indicates the high efficacy even when protecting the sensitive data in a relatively complex application.

7.4 Granularity Conflict Study

As discussed in Section 7.4, the choice of retaining the tag will cause over-tagging issue. We show the experiment result of the over-tagging ratio in Table 1.

The result shows that except rijndael, three other applications suffer from the granularity conflict. In particular, there is 0.033% sensitive data over-tagging for blowfish. That said, 0.033% of the protected data is supposed to be insensitive. The over-tagging result is 0.13% for zip30, and 5.12% for OpenSSL. We additionally estimate the cost of protecting the wrongly tagged data. The result is it imposes 0.14%, 1.04%, and 2.16% extra runtime overhead for blowfish, zip30, and OpenSSL, respectively.

7.5 Security Analysis

`Conch` aims to safeguard the confidentiality of sensitive data. The threat model is strong as the attacker is granted with arbitrary read and write primitive. Because `Conch` utilizes the tagged architecture to dynamically track the sensitive data and encrypt it before it is written into memory, the sensitive data cannot be leaked.

In the following, we will analyze the possible loophole due to the granularity and sensitivity conflict.

Granularity Conflict Our system retains the tag in the granularity conflict. Thus it will not remove tags for sensitive data. As a result, the security guarantees still hold, although unnecessary data may be protected.

Sensitivity Conflict We allow the developer to rectify the tag propagation with the ability to remove the memory tag

of a buffer to solve the sensitivity conflict issue (Section 5.2). In our threat model, the attacker cannot hijack the control flow so the tag removing instruction cannot be abused to bypass our protection.

8 Related Work

Selective Data Protection Instead of protecting all memory objects, selective data protection only focuses on partial targets, such as pointers, control-flow related variables, and the developer specified sensitive data. Some recent approaches achieve this by requiring the programmer to annotate sensitive memory objects in the source code. DataShield[6] and ConfLLVM[5] prepare dedicated memory region and additional bound checking for the annotated sensitive data. Tapti et al[23], similar to `Conch`, selectively encrypts the sensitive content before it is written into memory. However, the cryptographic transformations they designed are done by the instrumentation code instead of the hardware engine hence has poor performance. Ginseng[33] innovatively protects annotated sensitive data by allocating them to registers at compile time and only puts it into memory when task switching or interrupts occur. To defend against a compromised kernel, Ginseng leverages the Trusted Execution Environment (TEE) to manage encryption and decryption. These approaches bear disadvantages like high runtime overhead, coarse granularity, and the difficulty of deploying.

Tag-based Memory Protection Several systems utilize the tagged architecture for protection end. For example, lowRISC[19] uses tags to specify whether a memory address is readable or writable. HDFI[28] uses tags to achieve data-flow integrity. CHERI[32] uses tags to indicate if a memory address stores a valid fat pointer. Among the previous work, Morpheus[13] is closest to `Conch`. It uses two-bit domain tags to distinguish code, code pointers, data pointers, and normal data and further adopts the domain encryption defense to randomizes the representation of code, code pointers, and data pointers before they are written into memory. The difference is that `Conch` aims to protect the developer specified sensitive data, which is the superset of Morpheus's target. Additionally, `Conch` promises protection ever since the data is initialized and only consumes one-bit tags, comparatively more lightweight and easy to implement.

Information Flow Tracking (IFT) IFT frameworks based on whole system emulation[10] incur high performance penalties, making it unsuitable for the protection of running programs. Taintdroid[11], instead, leverages the architectural features of virtual-machine-based smartphones to enable efficient, system-wide taint tracking to monitor how third-party applications handle the private data of users. There are two main differences between `Conch` and Taintdroid. First, `Conch` takes advantage of tagged architecture to utilize hardware support taint analysis while Taintdroid is based on the virtual machine and needs to instrument the VM interpreter. Second, Taintdroid automatically detects and defines privacy-sensitive sources, such as GPS location and APP List. Hence it cannot work as flexibly as `Conch` to protect specific data that the developer expects. There are several general hardware architectures[9, 26, 27] that can be used to achieve dynamic IFT. `Conch` leverages a similar architecture compared with them but only requires one-bit tags, which is proven to be efficient[16].

9 Conclusion

In this paper, we revisit the technical challenges that were usually neglected by previous systems when applying selective data protection to real-applications, and propose corresponding solutions. Then we design and implement a prototype system for selective data protection and evaluate the overhead using the Spike simulator. The evaluation demonstrates the efficiency (less than 3% overhead with optimizations) and the security guarantees provided by our system.

10 Acknowlegements

The authors would like to thank the anonymous reviewers for their insightful comments that helped improve the presentation of this paper. This work was partially supported by the National Natural Science Foundation of China (grant No.61872438), Leading Innovative and Entrepreneur Team Introduction Program of Zhejiang (No. 2018R01005), the Fundamental Research Funds for the Central Universities (Zhejiang University NGICS Platform, K20200019). Any opinions, findings, and conclusions expressed in this paper are of the authors and do not necessarily reflect the views of funding agencies.

References

[1] CVE-2019-6840. Available from mitre. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-6840, 2020.

[2] CVE-2020-15203. Available from mitre. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-15203, 2020.

[3] Avanzi et al. The qarma block cipher family. *IACR Transactions on Symmetric Cryptology*, 2017.

[4] Bradbury et al. Tagged memory and minion cores in the lowrisc soc. *Memo, University of Cambridge*, 2014.

[5] Brahmakshatriya et al. Confllvm: A compiler for enforcing data confidentiality in low-level code. In *Proceedings of the Fourteenth EuroSys Conference 2019*, 2019.

[6] Carr et al. Datashield: Configurable data confidentiality and integrity. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, 2017.

[7] Chow et al. Understanding data lifetime via whole system simulation. In *USENIX Security Symposium*, 2004.

[8] Crandall et al. Minos: Control data attack prevention orthogonal to memory model. In *37th International Symposium on Microarchitecture*, 2004.

[9] Dhawan et al. Pump: a programmable unit for metadata processing. In *Proceedings of the Third Workshop on Hardware and Architectural Support for Security and Privacy*, 2014.

[10] Egele et al. Dynamic spyware analysis. 2007.

[11] Enck et al. Taintdroid: an information-flow tracking system for realtime privacy monitoring on smartphones. *ACM Transactions on Computer Systems*, 2014.

[12] Feustel et al. On the advantages of tagged architecture. *IEEE Transactions on Computers*, 1973.

[13] Gallagher et al. Morpheus: a vulnerability-tolerant secure architecture based on ensembles of moving target defenses with churn. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019.

[14] Greathouse et al. A case for unlimited watchpoints. *ACM SIGPLAN Notices*, 2012.

[15] Guthaus et al. Mibench: A free, commercially representative embedded benchmark suite. In *Proceedings of the fourth annual IEEE international workshop on workload characterization.*, 2001.

[16] Joannou et al. Efficient tagged memory. In *2017 IEEE International Conference on Computer Design*, 2017.

[17] Kang et al. Dta++: dynamic taint analysis with targeted control-flow propagation. In *NDSS*, 2011.

[18] Kannan et al. Decoupling dynamic information flow tracking with a dedicated coprocessor. In *2009 IEEE/IFIP International Conference on Dependable Systems & Networks*, 2009.

[19] Kwon et al. Low-fat pointers: compact encoding and efficient gate-level implementation of fat pointers for spatial safety and capability-based security. In *Proceedings of the 2013 ACM SIGSAC conference on Computer and Communications Security*, 2013.

[20] Li et al. Dramsim3: a cycle-accurate, thermal-capable dram simulator. *IEEE Computer Architecture Letters*, 2020.

[21] Liljestrand et al. {PAC} it up: Towards pointer integrity using {ARM} pointer authentication. In *28th {USENIX} Security Symposium ({USENIX} Security 19)*, 2019.

[22] Newsome et al. Dynamic taint analysis for automatic detection, analysis, and signaturegeneration of exploits on commodity software. In *NDSS*, 2005.

[23] Palit et al. Mitigating data leakage by protecting memory-resident sensitive data. In *Proceedings of the 35th Annual Computer Security Applications Conference*, 2019.

[24] Roessler et al. Protecting the stack with metadata policies and tagged hardware. In *2018 IEEE Symposium on Security and Privacy (SP)*, 2018.

[25] Shankar et al. Detecting format string vulnerabilities with type qualifiers. In *USENIX Security Symposium*, 2001.

[26] Shrobe et al. Trust-management, intrusion-tolerance, accountability, and reconstitution architecture (tiara). Technical report, 2009.

[27] Song et al. Towards general purpose tagged memory. In *Proceedings of the RISC-V Workshop*, 2015.

[28] Song et al. Hdfi: Hardware-assisted data-flow isolation. In *2016 IEEE Symposium on Security and Privacy (SP)*, 2016.

[29] Strackx et al. Breaking the memory secrecy assumption. In *Proceedings of the Second European Workshop on System Security*, 2009.

[30] Suh et al. Secure program execution via dynamic information flow tracking. *ACM Sigplan Notices*, 2004.

[31] Weiser et al. TIMBER-V: Tag-Isolated Memory Bringing Fine-grained Enclaves to RISC-V. 2019.

[32] Woodruff et al. The cheri capability model: Revisiting risc in an age of risk. In *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, 2014.

[33] Yun et al. Ginseng: Keeping secrets in registers when you distrust the operating system. In *NDSS*, 2019.

[34] Michael Hind. Pointer analysis: Haven't we solved this problem yet? In *Proceedings of the 2001 ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools and engineering*, 2001.

[35] SiFive Inc. Fu540-c000 manual v1p0. https://www.sifive.com/boards/hifive-unleashed, 2018.